Assignment 1.2

Professor: Richard Krasso

Student: Jonathan Cantu

Part I: BlueSky Apps DevOps Analysis

1. DevOps Principles

The problems at BlueSky Apps demonstrate weaknesses in all three DevOps principles of Flow, Feedback, and Continuous Learning. Of these, Flow and Feedback appear to be the most immediate problems. Developers commit code to a shared branch, but automated testing does not occur as part of the development process. Testing is postponed until the end of the sprint, creating a bottleneck where the QA engineer must manually test everything. Defects discovered at this point are more expensive and time-consuming to correct because developers may have already moved on to other work. Manual deployments create another interruption in Flow because releases depend on operations completing a slow and error-prone process.

Feedback is also severely limited. Developers do not receive immediate feedback when they introduce a defect, and the lack of centralized logging and monitoring means the organization may not know a production problem exists until a customer reports it. In both situations, feedback arrives too late to prevent additional work or customer impact.

Continuous Learning is also underdeveloped. Development and operations operate largely as separate groups and blame each other when failures occur. This environment discourages shared ownership and prevents the organization from learning effectively from failures. A healthier DevOps culture would treat an incident as an opportunity to understand what failed in the system or process and then improve it.

2. Automation Opportunities

I would begin by automating the processes that currently create the largest delays and opportunities for human error. Automated testing should be integrated into the development workflow so tests execute whenever developers make changes. Unit, integration, and eventually end-to-end testing would give developers faster feedback and allow QA to concentrate more on exploratory and higher-value testing rather than repeatedly validating basic functionality.

Deployment should also become a repeatable automated process. Instead of operations manually performing a series of deployment steps, those steps should be defined and executed consistently through an automated pipeline. The process should include validation before deployment and the ability to roll back when a release fails. This would improve both deployment speed and reliability.

Finally, BlueSky needs automated monitoring and centralized logging. Application errors, performance metrics, system health, and other operational information should be collected centrally with alerts for conditions requiring attention. The goal should be to identify problems internally before customers have to report them.

3. AI-Assisted Improvements

AI could strengthen this automation in several areas. The first is **AI-assisted test generation**. Developers could use AI to suggest test cases based on application code and requirements, including boundary conditions and unusual inputs they may not initially consider. These tests should still be reviewed by developers, but AI could reduce the time required to create broader test coverage and help address BlueSky's current testing bottleneck.

A second high-impact area is **anomaly detection and incident response**. Traditional monitoring generally depends on engineers defining thresholds for known conditions. AI-assisted monitoring could analyze logs and metrics to establish patterns of normal behavior and identify unusual activity that does not match those patterns. During an incident, AI could also correlate related logs, alerts, recent changes, and previous incidents to help engineers identify likely root causes. I would view AI in this role as an intelligent first responder rather than a replacement for the engineer. It can gather and analyze information quickly while the engineer maintains responsibility for decisions that could affect production.

4. Reflection on the Three Ways

These changes would reinforce all Three Ways. **Flow** would improve because automated testing and deployment would remove manual bottlenecks and allow smaller changes to move through the delivery process more consistently. **Feedback** would improve because developers would learn about failed tests shortly after making a change, while monitoring and anomaly detection would provide operations with much faster information about production behavior.

Most importantly, **Continuous Learning** would connect the other two principles. Test failures, deployment results, monitoring data, and production incidents all provide information the team can use to improve its processes. Developers, QA, and operations should review that information together instead of treating failures as belonging to one group. AI can help analyze the growing amount of information available to the team, but the team still has to apply engineering judgment and decide what should change. The end result would be a DevOps environment where BlueSky delivers smaller changes more frequently, receives feedback sooner, and continuously improves based on what it learns.

Part II: AI-Assisted Test Case Generation

I provided the following JavaScript function to ChatGPT and asked it to generate test cases, rather than complete unit tests:

```
function add(a, b)
{   return a + b; }
```

AI-Generated Test Cases

- `add(1, 2) -> 3`
- `add(0, 0) -> 0`
- `add(-1, 5) -> 4`
- `add(-5, -10) -> -15`

- `add(2.5, 3.7) -> 6.2`
- `add(1000000, 1) -> 1000001`
- `add(0, -5) -> -5`
- `add("1", 2) -> "12"`

Reflection

The AI-generated test cases seemed reasonable because they covered more than the most obvious case of adding two positive integers. It included zero, negative numbers, decimals, and large values, which provides better coverage of how the function might be used. The case that stood out to me was `add("1", 2)`, because JavaScript returns `"12"` rather than `3` due to type coercion. That is exactly the type of case that can reveal an assumption about the inputs even though the function itself is extremely simple. I would not automatically accept every AI-generated test case as necessary, but I would use the results to identify scenarios worth considering. This demonstrates where AI-assisted development can be valuable: it can quickly generate additional possibilities while the developer determines which ones are relevant to the application's requirements. AI therefore becomes another tool for improving test coverage and shortening the feedback cycle rather than replacing engineering judgment.